

Python Programming

Assignment - 4

Q) What is the difference between List and Tuple in Python? Explain in terms of:

- Mutability
- Memory
- Performance
- Use cases

Ans:

• Mutability :
 a) **List:** Lists are mutable, meaning we can modify, add, change or remove elements after creation without changing the identity of the list.

b) **Tuple:** Tuples are immutable. Once created, their elements cannot be changed, added to or deleted.

• Memory :

a) **List:** Consumes more memory, because interpreter leaves extra space in memory because it expects you to add more items later.

b) **Tuple:** Uses less memory, it allocates the exact space needed because its size is fixed.

• Performance :

a) **List:** Slow, because it is flexible and has extra memory overhead, it takes Python a bit more time to create and process.

b) **Tuple:** Faster, because it is simple and unchangeable, Python can process it much quicker.

- Use cases:
- List: Used for changing data (eg, a to-do list where we add and cross off tasks)
 - Tuple: Used for fixed data that must stay safe from accidental changes.

2) Why are tuples faster than lists?
In which real-world scenario would you prefer a tuple over a list?

Ans: Tuples are faster than lists because tuples cannot be changed after they are created, while lists can. Python has to keep extra information for lists so they can grow, shrink, insert, and delete items. Tuples don't need that extra work, so they use less memory and are slightly faster to create and access.

3) Predict the output:

lst: [10, 20, 30]

tpl: (10, 20, 30)

lst[0] = 100

tpl[0] = 100

Which line will raise an error & why?

Ans: The line `tpl[0] = 100` raises the error. This is because Tuples are immutable, which means they are strictly read-only once created. It blocks us from changing any value inside a tuple. So trying to assign new value causes a `TypeError`.

4) Explain why strings are immutable in Python.
What happens internally when you modify a string variable?

Ans: A string is immutable, meaning once a string object is created, its contents cannot be changed. This is because, Python can safely reuse string objects. Frequently used strings can be cached internally. Also strings can be used as dictionary keys.

Since we can't alter a string object,
code:

```
text = "Hello"
```

```
text = text + "World"
```

It looks like we changed text, but internally, Python never modified the original "Hello" object. Instead, it performed a rebinding process:

Step 1 (text = "Hello"): It creates a string object "Hello" at a specific memory address and points the name tag text to it.

Step 2 (text + "World"): When you add "World", Python looks at the old text, creates a completely brand-new string object containing "Hello World" at a new memory address.

Step 3 (Rebinding): Python detaches the name tag & text from Address 101 and hooks or points it onto the new object at address 202.

Conclusion: We can never modify a string object itself. When we change a string variable, we are simply throwing away the old string and making a brand new one at a new spot in memory.

5) Predict the output:

```
s = "Python"
```

```
print(id(s))
```

```
s = s + "3"
```

```
print(id(s))
```

Explain the reason for change/no change in id().

Ans:

Output:

488256

57408

The id() number will change. There will be two different id's, so there is change.

Think of id() like a home address in "memory", and the variable s like a name tag.

a) s = "Python": Python builds a house called "Python" at Address A, and sticks the name tag s on it.

b) s = s + "3": In python, strings are immutable (cannot just tack a "3" onto the roof of old house).

- Instead, Python builds a completely new house called "Python3" at a completely new address (Address B).

- It then peels the name tag s off the first house and sticks it onto the new house.

Because the name tag s is now pointing to a new house, the second print(id(s)) gives us a new address number.

6) What is ~~dict~~ dictionary in Python?

Explain:

- Key-value pair
- Why keys must be immutable
- Why duplicate keys are not allowed

Ans: ~~Dict~~ Dictionary (dict) stores data in key-value pairs, making data retrieval fast and efficient.

A key-value pair is a way to store data where a specific key acts like descriptive label, and the value is the actual data attached to that label.

In Python, keys dictionary keys must be immutable (unchangeable, like strings, numbers or tuples). They cannot be mutable objects like lists.

Dictionary keys must be completely unique. If you try to add a duplicate key, it will not give error, but it will overwrite the older stored value with the new one.

7) Predict the output:

```
d = {1: "One", 1: "ONE", 2: "Two"}
```

```
print(d)
```

Why does this happens?

Ans: Output:

```
{1: "ONE", 2: "Two"}
```


This happens because Duplicate keys are not allowed in a dictionary. This is because, from the above code we get to know that or it travels like;

- i) Python starts creating the dictionary and sees `1: "One"`. It saves the key 1 with the value "One".
- ii) Next, it encounters `1: "ONE"`. It notices that the key 1 already exists in the dictionary.
- iii) Instead of creating a second key, Python simply overwrites the old value ("one") with the newest value provided ("ONE").
- iv) Finally, it adds `2: "Two"` without any issues since 2 is a unique value.

8) What is the range data type in Python?
How is it different from a list of numbers?

Ans: The range data type represents a sequence of numbers and is commonly used in loops for iteration.

A list actually creates and holds every single number in computer memory.

ex: `[1, 2, 3, 4, 5]`

A range does not create any numbers upfront. It only stores three simple rules: where to start, where to stop and how to count. It calculates each number only when you ask for it.

ex: `range(1, 6)`

9) Predict the output:

```
r = range(5)
print(r)
print(list(r))
```

Ans: Output:
range(0, 5)
[0, 1, 2, 3, 4]

10) What is the difference between:

range(1, 10)

range(1, 10, 2)

Explain the parameters.

Ans: The difference is:

range(1, 10)

- It starts 1, stops before 10, and counts by 1 (default)
- The numbers generate: 1, 2, 3, 4, 5, 6, 7, 8, 9

range(1, 10, 2)

- It starts 1, stops before 10, and counts by 2.
- The numbers generate: 1, 3, 5, 7, 9

Parameters:

The range() function can take up to three parameters:

range(start, stop, step)

- start: The number where the counting begins
- stop: The number where the counting ends (n-1).
- step: How much to add or skip each time. If it is not specified it by default was a step of 1.